Storage and filesystems

Choose block, file or object storage, manage ext4 and XFS, LVM and RAID, NFS, iSCSI and Ceph, and diagnose slow or full disks.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Block, file and object storage (13 min)
2. Filesystems, mounts and fstab (14 min)
3. LVM and RAID (15 min)
4. Network storage: NFS, iSCSI and Ceph (14 min)
5. Performance, backups and troubleshooting (15 min)
6. Backup and DR: strategy, the 3-2-1 rule and retention (12 min)
7. Backup and DR: database backups and point-in-time recovery (13 min)
8. Backup and DR: Kubernetes, etcd and volume snapshots (12 min)
9. Backup and DR: testing restores and verifying backups (11 min)
10. Backup and DR: disaster recovery patterns and failover (12 min)
11. Backup and DR: ransomware, immutable backups and recovery runbooks (12 min)
12. Storage benchmarking with fio (12 min)
Cheat sheet

Lesson 1. Block, file and object storage

13 minutes. Tell the three kinds of storage apart, and learn the numbers that matter: capacity, IOPS, throughput and latency.

Everything an application saves ends up on some kind of storage. The kind decides what you can do with it, how fast it is, and what breaks when it fills up.

What you will learn

- Block, file and object storage and when to use each
- IOPS, throughput and latency, and how they differ
- The main device types: HDD, SSD and NVMe
- How to measure a disk from the command line

Three kinds

Kind	You see	Examples	Good for
Block	A raw disk you format yourself	Local disk, SAN LUN, AWS EBS, Azure managed disk	Databases, boot disks, virtual machines
File	Folders and files shared over a network	NFS, SMB, AWS EFS, Azure Files	Shared content, home folders
Object	Whole objects with keys, over HTTP	AWS S3, Azure Blob, Ceph RGW	Backups, media, logs, data lakes

A block device has no idea what is in it. A **filesystem** on top gives it names and folders. Object storage has no folders, only keys that may contain slashes, and objects are replaced as a whole, not edited in place.

The numbers

- **Capacity**: how much it holds.
- **IOPS**: input and output operations per second. Many small reads and writes, such as a database, are IOPS limited.
- **Throughput**: megabytes per second. Large sequential copies, such as backups, are throughput limited.
- **Latency**: the time for one operation. A database commit waits for it.

A disk that does 100 000 IOPS with 4 KB operations moves only about 400 MB per second. The same disk copying large blocks may move 2 000 MB per second. Always ask which number a workload needs.

Device types

- **HDD** (spinning): cheap per terabyte, a few hundred IOPS, milliseconds of latency.
- **SSD (SATA or SAS)**: tens of thousands of IOPS, latency well under a millisecond.
- **NVMe**: flash on the PCIe bus, hundreds of thousands of IOPS and very low latency.

Cloud disks add limits of their own: a volume may have a baseline and a burst. When the burst credit is spent, it slows to the baseline.

MARGA by EknathaLabs marga.eknathalabs.com

Measuring

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  100G  0 disk
|-sda1   8:1    0    1G  0 part /boot
`-sda2   8:2    0   99G  0 part /
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        98G   71G   22G  77% /
$ fio --name=randread --rw=randread --bs=4k --size=1G --runtime=10 --time_based --iodepth=16
  read: IOPS=21.4k, BW=83.7MiB/s
```

`fio` is the standard tool for testing, and it should be run on a test disk, never on a disk with live data in a way that writes over files you need.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Block, file and object storage fits together

Part	What it does	What goes wrong
Three kinds	A block device has no idea what is in it.	Treating object storage like a filesystem that supports small in-place edits.
The numbers	Capacity: how much it holds.	Testing with fio on a path that has a cache and getting numbers that are too good.
Device types	HDD (spinning): cheap per terabyte, a few hundred IOPS, milliseconds of latency.	Choosing a volume by price per terabyte without checking IOPS.
Measuring	fio is the standard tool for testing, and it should be run on a test disk, never on a disk with live data in a way that writes over files you need.	Measuring fio is the standard tool for testing, and it should be run on a test disk, never on a disk with live data in a way that writes...

Flow: Three kinds to The numbers / The numbers to Device types / Device types to Measuring

Incident: The cheap disk that made the database slow

Incident: *A report job that took ten minutes on the old server took two hours after migration to a cheaper volume type.*

```
$ iostat -x 5 2 | grep -E 'Device|nvme|sda'
Device   r/s    w/s   rkB/s  await  %util
sda      95.0   12.0  1520   42.10  99.80
```

The new volume was a throughput-oriented HDD type. The database made many small random reads, so the disk was at 99.8 percent busy and each read waited 42 ms. The old disk was SSD. Moving the data to an SSD volume brought the report back to ten minutes. Capacity and price were not the issue; the access pattern was.

Common mistakes

- Choosing a volume by price per terabyte without checking IOPS
- Treating object storage like a filesystem that supports small in-place edits
- Testing with fio on a path that has a cache and getting numbers that are too good
- Ignoring cloud burst limits until a busy hour

Check yourself

Which kind of storage is a raw disk that you format yourself?

Answer: Block storage

A workload does many small random reads. Which number matters most?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: IOPS

What does a filesystem add on top of a block device?

Answer: Names, folders and file metadata

Why can a cloud volume slow down after a busy period?

Answer: Its burst allowance is used up and it falls to the baseline

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Filesystems, mounts and fstab

14 minutes. Compare ext4, XFS and others, mount a disk safely, and avoid the fstab mistakes that stop a server from booting.

A filesystem turns a block device into folders and files. Picking one is a smaller choice than most people expect, but mounting it correctly matters a great deal.

What you will learn

- ext4, XFS, btrfs and ZFS in a few sentences each
- Inodes, journals and why a disk can be full with space free
- Mounting by UUID and writing a safe fstab line
- What to do when a filesystem needs repair

Common filesystems

- **ext4**: the long-time Linux default. Stable, well understood, can grow and, with care, shrink.
- **XFS**: strong with large files and many parallel writes, the default on Red Hat family systems. It can grow while mounted but cannot shrink.
- **btrfs**: copy-on-write with snapshots and checksums, used by some distributions.
- **ZFS**: combines volume management and filesystem, with checksums, snapshots and replication. Its licence keeps it outside the main Linux kernel.

Pick what your distribution supports and your team knows. Most production trouble comes from full disks and bad mounts, not from the choice between ext4 and XFS.

Inodes and journals

An **inode** holds a file's metadata: owner, permissions, size and where its data blocks are. A filesystem has a fixed number of inodes on ext4. A disk with millions of tiny files can run out of inodes while plenty of bytes are free, and every write fails with "No space left on device".

```
$ df -h /var
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb1       200G   61G  129G  32% /var
$ df -i /var
Filesystem     Inodes   IUsed IFree IUse% Mounted on
/dev/sdb1     13107200 13107200     0  100% /var
```

A **journal** records changes before making them, so after a crash the filesystem replays the journal instead of checking everything.

Mounting

```
$ blkid /dev/sdb1
/dev/sdb1: UUID="3f1c0d7e-8a42-4b9b-9c11-2e6a5d0f1a77" TYPE="ext4"
$ sudo mkdir -p /data
$ sudo mount /dev/sdb1 /data
$ findmnt /data
TARGET SOURCE    FSTYPE OPTIONS
/data  /dev/sdb1 ext4   rw,relatime
```

Device names such as `/dev/sdb1` can change between boots. In `/etc/fstab`, refer to the disk by **UUID**:

```
UUID=3f1c0d7e-8a42-4b9b-9c11-2e6a5d0f1a77  /data  ext4  defaults,nofail  0  2
```

The six fields are device, mount point, type, options, dump (leave 0) and fsck order. `nofail` lets the server boot even if the disk is missing. Without it, a missing data disk can drop the machine into emergency mode. After editing fstab, always run `sudo mount -a`: it mounts everything in the file and shows an error now, not at the next reboot.

MARGA by EknathaLabs marga.eknathalabs.com

Repair

An unclean shutdown usually needs only a journal replay. When `fsck` is needed, run it on an unmounted filesystem and read what it will change first. Never run it on a mounted disk. For a disk with hardware errors, copy what you can with a tool such as `ddrescue` before repairing.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Filesystems, mounts and fstab fits together

Part	What it does	What goes wrong
Common filesystems	ext4: the long-time Linux default. Stable, well understood, can grow and, with care, shrink.	Running fsck on a mounted filesystem.
Inodes and journals	An inode holds a file's metadata: owner, permissions, size and where its data blocks are.	Editing fstab and not testing with mount -a.
Mounting	Device names such as /dev/sdb1 can change between boots.	Referring to disks by /dev/sdX in fstab.
Repair	An unclean shutdown usually needs only a journal replay.	Looking only at df -h and forgetting df -i.

Flow: Common filesystems to Inodes and journals / Inodes and journals to Mounting / Mounting to Repair

Incident: The server that did not come back

Incident: *After a routine reboot a server stayed in emergency mode and the team had no console access plan.*

```
[ TIME ] Timed out waiting for device /dev/disk/by-uuid/3f1c0d7e-8a42-4b9b-9c11-2e6a5d0f1a7.
[DEPEND] Dependency failed for /data.
You are in emergency mode.
```

Someone had retyped a UUID into fstab and dropped the last character, and the line had no `nofail`. The system waited for a device that did not exist. Fixing the UUID in `/etc/fstab` from the emergency shell and rebooting restored service. The team now copies UUIDs from `blkid`, adds `nofail` for data disks, and runs `mount -a` before leaving the server.

Common mistakes

- Referring to disks by `/dev/sdX` in fstab
- Editing fstab and not testing with `mount -a`
- Looking only at `df -h` and forgetting `df -i`
- Running fsck on a mounted filesystem

Check yourself

Why use UUIDs in /etc/fstab?

Answer: Device names such as /dev/sdb can change between boots

A disk shows 32 percent used but writes fail with No space left. What should you check?

Answer: df -i, because the inodes may be used up

What does the nofail option do in fstab?

Answer: Lets the system boot even if that disk is missing

Which command tests an fstab edit without rebooting?

Answer: sudo mount -a

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. LVM and RAID

15 minutes. Grow volumes online with LVM, understand RAID levels and their trade-offs, and handle a failed disk calmly.

A server's data will outgrow its disk. LVM lets you add space without moving data, and RAID lets a disk fail without losing it. They solve different problems and are often used together.

What you will learn

- Physical volumes, volume groups and logical volumes
- Growing a volume and its filesystem online
- RAID 0, 1, 5, 6 and 10 and when each fits
- What to do when a disk fails

LVM

LVM (logical volume manager) adds a layer between disks and filesystems.

- A **physical volume (PV)** is a disk or partition given to LVM.
- A **volume group (VG)** is a pool made of one or more PVs.
- A **logical volume (LV)** is a slice of the pool that you format and mount.

```
$ sudo pvs
  PV         VG   Fmt  Attr PSize   PFree
  /dev/sdb   vg0  lvm2 a--  500.00g 120.00g
$ sudo vgs
  VG   #PV #LV #SN Attr   VSize   VFree
  vg0    1   2   0 wz--n- 500.00g 120.00g
$ sudo lvs
  LV   VG  Attr       LSize
  data vg0 -wi-ao---- 300.00g
  logs vg0 -wi-ao---- 80.00g
```

To grow the `data` volume by 50 GB, the pool needs free space (here 120 GB is free) and the filesystem must grow too:

```
$ sudo lvextend -r -L +50G /dev/vg0/data
  Size of logical volume vg0/data changed from 300.00 GiB to 350.00 GiB.
  Logical volume vg0/data successfully resized.
```

The `-r` option resizes the filesystem as well. Without it the volume is larger but `df` shows the old size, because the filesystem still thinks it has 300 GiB. In that case run `resize2fs` (ext4) or `xfs_growfs` (XFS). If the pool has no free space, add a new disk with `pvcreate` and `vgextend` first. XFS can grow but not shrink.

RAID

RAID combines disks for speed, safety or both. It protects against disk failure, not against deletion, corruption or fire. It is not a backup.

Level	Disks	Survives	Notes
0	2+	Nothing	Speed only: one failure loses everything
1	2	One disk	Mirror, half the capacity
5	3+	One disk	Parity, slow to rebuild on big disks
6	4+	Two disks	Double parity, better for large disks
10	4+	One per mirror	Mirrors striped, fast and safe

MARGA by EknathaLabs
marga.eknathalabs.com

Rebuilding a large disk takes many hours, and the surviving disks work hard the whole time. With RAID 5 on big disks the chance of a second failure during rebuild is real, which is why RAID 6 or RAID 10 is preferred for large arrays.

```
$ cat /proc/mdstat
md0 : active raid1 sdb1[0] sdc1[1](F)
      976630336 blocks [2/1] [U_]
$ sudo mdadm --detail /dev/md0 | grep -E 'State|Failed'
          State : clean, degraded
 Failed Devices : 1
```

`[U_]` shows one of two members is missing: the array is **degraded** but working.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How LVM and RAID fits together

Part	What it does	What goes wrong
LVM	LVM (logical volume manager) adds a layer between disks and filesystems.	Extending an LV and forgetting to grow the filesystem.
RAID	RAID combines disks for speed, safety or both.	RAID 5 on very large disks with no spare.
Incident: Two failures, one night	One disk had failed a week before, the alert went to an unmonitored mailbox, and the rebuild began only after a second disk showed errors.	Treating RAID as a backup.

Flow: LVM to RAID / RAID to Incident: Two failures, one night

Incident: Two failures, one night

Incident: *A RAID 5 array lost a second disk while rebuilding the first.*

```
$ cat /proc/mdstat
md0 : active raid5 sde1[4](S) sdd1[3](F) sdc1[2] sdb1[1](F) sda1[0]
      [5/3] [U_U__]
```

One disk had failed a week before, the alert went to an unmonitored mailbox, and the rebuild began only after a second disk showed errors. With two members gone the array stopped. The team restored from last night's backup and rebuilt the array as RAID 6 with working alerts. RAID kept the first failure harmless; nothing in the design covered a second one that nobody knew about.

Common mistakes

- Treating RAID as a backup
- Extending an LV and forgetting to grow the filesystem
- No alert on a degraded array
- RAID 5 on very large disks with no spare

Check yourself

What does lvextend -r do?

Answer: Grows the logical volume and its filesystem together

Which RAID level survives two disk failures?

Answer: RAID 6

Why is RAID not a backup?

Answer: It does not protect against deletion, corruption or site loss

What does [U_] in /proc/mdstat show?

Answer: One member of a two-disk mirror is missing

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Network storage: NFS, iSCSI and Ceph

14 minutes. See how storage is shared over a network with NFS and iSCSI, and what a distributed system such as Ceph adds.

When many servers need the same data, or a server's disk must outlive the server, storage moves onto the network.

What you will learn

- NAS and SAN, and how NFS and iSCSI fit
- Mounting NFS and the options that matter
- iSCSI initiators and targets
- What Ceph is and how it keeps copies

NAS and SAN

- **NAS** (network attached storage) shares **files**. Clients see folders. NFS and SMB are NAS protocols.
- **SAN** (storage area network) shares **blocks**. Clients see a disk and format it. iSCSI and Fibre Channel are SAN protocols.

NFS

NFS lets a server export a folder that clients mount like a local one.

```
$ showmount -e nas01
Export list for nas01:
/exports/media 10.0.0.0/24
$ sudo mount -t nfs4 -o rw,hard,timeo=600 nas01:/exports/media /mnt/media
$ findmnt /mnt/media
TARGET     SOURCE            FSTYPE OPTIONS
/mnt/media nas01:/exports/media nfs4  rw,relatime,hard,timeo=600
```

Options worth knowing:

- **hard** keeps retrying when the server is unreachable, so applications wait instead of getting errors that corrupt data. **soft** gives up and returns an error, which can silently lose writes.
- File ownership uses numbers (UID and GID). If the user is 1001 on the client and 1002 on the server, permissions look wrong. Either keep IDs in sync or use a central directory.
- A client that cannot reach the server may freeze commands such as `ls` or `df` on that mount. Applications that hang in uninterruptible sleep (state `D`) are a common sign.

iSCSI

iSCSI carries SCSI commands over ordinary IP. The server that offers a disk is the **target**, and the client is the **initiator**.

```
$ sudo iscsiadm -m discovery -t sendtargets -p 10.0.1.20
10.0.1.20:3260,1 iqn.2026-01.example:storage.lun1
$ sudo iscsiadm -m node -T iqn.2026-01.example:storage.lun1 -p 10.0.1.20 --login
$ lsblk | grep sdc
sdc      8:32   0  200G  0 disk
```

The new disk is raw; you create a filesystem on it. Use a separate network or VLAN for iSCSI, and **multipath** (two paths to the storage) so one switch failure does not remove the disk.

Ceph

Ceph is a distributed storage system that runs on ordinary servers. It offers block (RBD), file (CephFS) and object (RGW, S3 compatible) storage on the same cluster.

MARGA by EknathaLabs
marga.eknathalabs.com

- **OSDs** store the data, one per disk.
- **Monitors** keep the cluster map and need a majority to agree.
- Data is split into objects placed by an algorithm called **CRUSH**, so no central table is needed.
- Each object has copies on different hosts (commonly 3), or is spread with **erasure coding**, which uses less space but costs CPU and repair work.

```
$ sudo ceph -s
  cluster:
    health: HEALTH_WARN
            1 osds down
  data:
    pools:   3 pools, 128 pgs
    usage:   4.1 TiB used, 18 TiB / 22 TiB avail
    pgs:     128 active+undersized+degraded
```

A down OSD leaves data with fewer copies than intended, shown as **degraded**. Ceph rebuilds copies on other disks automatically. Replacing the disk quickly keeps the cluster safe, and a full cluster is a serious failure: Ceph stops writes at about 95 percent full to protect itself.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Network storage: NFS, iSCSI and Ceph fits together

Part	What it does	What goes wrong
NAS and SAN	NAS (network attached storage) shares files.	iSCSI on the same network as general traffic, with no multipath.
NFS	NFS lets a server export a folder that clients mount like a local one.	Using soft mounts for data that matters.
iSCSI	iSCSI carries SCSI commands over ordinary IP.	Different user IDs on client and server.
Ceph	Ceph is a distributed storage system that runs on ordinary servers.	Letting a Ceph cluster fill past 85 percent.

Flow: NAS and SAN to NFS / NFS to iSCSI / iSCSI to Ceph

Incident: The share that froze every server

Incident: *One NFS server was rebooted, and thirty application servers hung within a minute.*

```
$ ps -eo pid,stat,wchan:20,cmd | grep ' D '
2231 D    nfs_wait_bit_killable  /opt/app/report-worker
2240 D    nfs_wait_bit_killable  df -h
```

The applications were mounted with `hard`, so they waited for the server, which is the safe choice for data but blocks everything. Because even `df -h` touched the mount, monitoring checks hung too. The NFS server came back after 12 minutes and everything resumed. The fix was to make the exports highly available, and to stop monitoring scripts calling `df` on network mounts without a timeout.

Common mistakes

- Using soft mounts for data that matters
- Different user IDs on client and server
- iSCSI on the same network as general traffic, with no multipath
- Letting a Ceph cluster fill past 85 percent

Check yourself

Which is a SAN protocol?

Answer: iSCSI

What does a hard NFS mount do when the server is unreachable?

Answer: It keeps retrying so applications wait instead of getting errors

In iSCSI, what is the target?

Answer: The server that offers the disk

What does a degraded Ceph placement group mean?

Answer: It has fewer copies than intended

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Performance, backups and troubleshooting

15 minutes. Find why a disk is slow or full, understand df versus du, and design backups that you have actually restored.

Storage faults come in a few shapes: the disk is full, the disk is slow, the disk is gone. This lesson gives you a method for each and ends with the part that matters most: backups you have tested.

What you will learn

- Reading iostat and iowait
- Finding what fills a disk, including deleted files that are still open
- Backup types, the 3-2-1 rule, and restore tests
- A short troubleshooting order

A slow disk

```
$ iostat -x 2 2
Device   r/s    w/s   rkB/s   wkB/s  await  aqu-sz  %util
sda      12.0  480.0  96.0   30720.0  85.3   41.2   100.0
$ top -bn1 | grep Cpu
%Cpu(s):  3.0 us,  2.0 sy,  0.0 ni, 12.0 id, 83.0 wa
```

- **await** is the average wait per request in milliseconds. High await with high `%util` means the device is saturated.
- **aqu-sz** (queue size) shows how many requests are waiting.
- **iowait** (`wa`) is CPU time spent idle while waiting for disk. A CPU that is 83 percent iowait is not busy; it is waiting.

Next, find who is doing the I/O: `iotop` or `pidstat -d 2` name the process. Then decide whether to slow the writer, move it, or move to faster storage.

A full disk

```
$ df -h /var
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        50G   50G     0 100% /var
$ sudo du -xh /var --max-depth=2 | sort -h | tail -4
1.2G    /var/lib
2.5G    /var/cache
4.1G    /var/log/app
8.0G    /var/log
```

`du` adds up the files it can see, `df` reports what the filesystem has allocated. Here `du` finds only 8 GB of 50 GB. The difference is often **a deleted file that a process still holds open**. The name is gone, so `du` does not see it, but the space is not freed until the process closes it.

```
$ sudo lsof +L1 | head -3
COMMAND  PID USER  FD  TYPE  SIZE/OFF NLINK NAME
java    1822 app   7w  REG   40G       0     /var/log/app/old.log (deleted)
```

`NLINK 0` and `(deleted)` identify the culprit. Restarting the process frees the space. If you cannot restart it, truncating through the file descriptor (`: > /proc/1822/fd/7`) frees the space without a restart, but know what you are discarding. The lasting fix is log rotation that signals the program to reopen its log.

Also check inodes with `df -i`, and file systems mounted over a directory that hides files beneath it.

Backups

- A **full** backup copies everything. An **incremental** copies changes since the last backup of any kind, and a **differential** copies changes since the last full one.

MARGA by EknathaLabs
marga.eknathalabs.com

- **3-2-1 rule**: three copies, on two kinds of media, one off-site.
- **Snapshots** are fast and local; they complement backups but do not replace them.
- **Immutable** or write-once backups protect against ransomware that deletes the backups it can reach.
- Define **RPO** (how much data you can lose) and **RTO** (how long you can be down). They set how often you back up and how fast you must restore.

A backup you have not restored is a hope. Schedule a restore test, restore to a scratch machine, and check that the application starts and data looks right.

Troubleshooting order

1. Is the device present and healthy? `lsblk`, `dmesg`, SMART data.
2. Is it mounted as expected? `findmnt`, `mount`.
3. Is there space and are there inodes? `df -h`, `df -i`.
4. Is it slow? `iostat -x`, then find the process.
5. Is a network dependency hanging? Look for `D` state processes on NFS mounts.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Performance, backups and troubleshooting fits together

Part	What it does	What goes wrong
A slow disk	await is the average wait per request in milliseconds.	Backups with no restore test, or stored on the same disk.
A full disk	du adds up the files it can see, df reports what the filesystem has allocated.	Deleting random files to free space instead of finding the cause.
Backups	A full backup copies everything. An incremental copies changes since the last backup of any kind, and a differential copies changes since the last full one.	- Snapshots are fast and local; they complement backups but do not replace them.
Troubleshooting order	Is the device present and healthy?. lsblk, dmesg, SMART data.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: A slow disk to A full disk / A full disk to Backups / Backups to Troubleshooting order

Incident: The log that ate the disk

Incident: *A disk alert fired at 100 percent, but the team could not find the files using the space.*

```
$ sudo du -sh /var/log
3.4G    /var/log
$ df -h /var/log
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        50G   50G     0 100% /var/log
$ sudo lsof +L1 | grep deleted
java  1822 app  7w  REG  44G  0  /var/log/app/app.log (deleted)
```

A logrotate job had removed `app.log` while the Java process kept writing to it. The 44 GB file had no name but still took the space. Restarting the service released the space and a `copytruncate` option was added to the rotation. The team had spent an hour looking for files that no longer existed by name.

Common mistakes

- Trusting `du` alone when `df` disagrees
- Deleting random files to free space instead of finding the cause

MARGA by EknathaLabs marga.eknathalabs.com

- Backups with no restore test, or stored on the same disk
- Reading iowait as CPU load

Check yourself

df shows 100 percent used but du finds only a small total. What is a likely cause?

Answer: A deleted file still held open by a process

What does high iowait mean?

Answer: The CPU is idle while waiting for disk requests

What does the 3-2-1 backup rule say?

Answer: Three copies, on two kinds of media, one off-site

Why test restores?

Answer: A backup that has not been restored is unproven

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Backup and DR: strategy, the 3-2-1 rule and retention

12 minutes. Design a backup strategy from recovery objectives, apply the 3-2-1 rule, choose full, incremental and snapshot methods, and set retention that matches business needs.

A backup is only useful if it lets you restore what you need, within the time you can afford, from a copy that survived the same disaster. Strategy starts from those requirements, not from the tool.

What you will learn

- Define what to protect and how fast you must recover
- The 3-2-1 rule and its modern variations
- Full, incremental, differential and snapshot backups
- Retention, encryption and cost

Start from objectives

For each system write down the **RPO** (how much data loss is acceptable) and the **RTO** (how long recovery may take). A reporting database with an RPO of 24 hours can use a nightly backup. A payments database with an RPO of one minute needs continuous log shipping or replication in addition. List what must be backed up: data, but also configuration, infrastructure code, secrets, certificates, DNS zones, and the runbooks and credentials needed to restore. A restore that needs a password stored only inside the system that failed is not a plan.

The 3-2-1 rule

Keep **3** copies of your data (the original plus two backups), on **2** different kinds of media or services, with **1** copy off-site (another region, account or provider). Modern advice adds: **1** copy that is offline or immutable, and **0** errors after verified restores (3-2-1-1-0). The point is independence: a failure, a mistake or an attacker should not be able to reach all copies by the same path. A replica is not a backup, because it copies a deletion or corruption within seconds.

Methods

- **Full**: a complete copy. Simple to restore, but large and slow.
- **Incremental**: only changes since the last backup of any kind. Small and fast to take, but a restore needs the whole chain.
- **Differential**: changes since the last full. Middle ground.
- **Snapshots**: point-in-time copies in the storage layer. Very fast, but usually stored near the source, so they are not a complete answer on their own.
- **Deduplicating tools** (restic, borg, pgBackRest, cloud backup services) store changed blocks once and make frequent backups cheap.

```
$ restic -r s3:s3.amazonaws.com/example-backups init
$ restic -r s3:s3.amazonaws.com/example-backups backup /srv/data --tag nightly
$ restic -r s3:s3.amazonaws.com/example-backups snapshots
$ restic -r s3:s3.amazonaws.com/example-backups forget --keep-daily 7 --keep-weekly 4 --keep-month \
    ly 12 --prune
```

Retention and cost

Keep recent backups often and older ones rarely: for example 7 daily, 4 weekly and 12 monthly, with legal or audit requirements overriding. Longer retention helps you find slow corruption and mistakes noticed late, but it costs storage and increases the amount of personal data you hold. Encrypt backups, keep the keys separately from the data (and back up the keys safely too), restrict who can delete, and tag backups with owner and retention.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: strategy, the 3-2-1 rule and... fits together

Part	What it does	What goes wrong
Start from objectives	For each system write down the RPO (how much data loss is acceptable) and the RTO (how long recovery may take).	Backing up data but not configuration, keys and runbooks.
The 3-2-1 rule	Keep 3 copies of your data (the original plus two backups), on 2 different kinds of media or services, with 1 copy off-site (another region, account or provider).	Storing backups where the same credentials can delete them.
Methods	Full: a complete copy. Simple to restore, but large and slow.	Treating replication as backup.
Retention and cost	Keep recent backups often and older ones rarely: for example 7 daily, 4 weekly and 12 monthly, with legal or audit requirements overriding.	Keeping encryption keys only inside the system being protected.

Flow: Start from objectives to The 3-2-1 rule / The 3-2-1 rule to Methods / Methods to Retention and cost

Incident: The backup in the same account

> **Incident:** *After an administrator's credentials were stolen, the attacker deleted a production database and, with the same access, its backups.*

```
$ aws s3 ls s3://prod-db-backups/
$ aws s3api list-object-versions --bucket prod-db-backups --max-items 3
```

Backups lived in the same cloud account, in a bucket the same administrators could empty. The company recovered only from a six-week-old export held by a partner. Afterward it copied backups into a separate account that production credentials could not write to or delete from, enabled object versioning with a retention lock, and added alerts on backup deletion.

Common mistakes

- Treating replication as backup
- Storing backups where the same credentials can delete them
- Backing up data but not configuration, keys and runbooks
- Keeping encryption keys only inside the system being protected

Check yourself

What does the 3-2-1 rule say?

Answer: Three copies, on two kinds of storage, with one copy off-site

Why is a replica not a backup?

Answer: It faithfully copies deletions and corruption almost immediately

What does RPO decide?

Answer: How often you must capture data

Which restore is slowest if you take daily incremental backups after a weekly full?

Answer: One at the end of the week, needing the full plus all increments

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Backup and DR: database backups and point-in-time recovery

13 minutes. Take consistent database backups, archive the write-ahead log for point-in-time recovery, and rehearse restoring to just before a mistake.

Copying database files while the database runs does not give a usable backup. Databases need a consistent snapshot plus a record of changes, so you can go back to a moment, such as one minute before the bad delete. This lesson uses PostgreSQL for examples, and the same ideas apply to MySQL binary logs and other engines.

What you will learn

- Logical and physical backups, and when each fits
- Base backups plus continuous log archiving
- Point-in-time recovery to a chosen time
- Practical tooling and what to monitor

Logical or physical

A **logical backup** (`pg_dump`) exports schema and data as SQL or an archive. It is portable across versions and good for single databases or small data, but restoring is slow for large data and it captures a single moment only. A **physical backup** copies the data files together with the write-ahead log (WAL), which is faster to restore at scale and enables point-in-time recovery.

```
$ pg_dump -Fc -d orders -f /backup/orders-2026-03-04.dump
$ pg_restore -d orders_restored --jobs 4 /backup/orders-2026-03-04.dump
$ pg_basebackup -h db1 -U replicator -D /backup/base -Ft -z -P
```

Archive the WAL

Every change goes to the WAL first. If you archive each WAL segment to safe storage, you can replay changes on top of a base backup up to any moment.

```
# postgresql.conf (shown as shell for the check): archive WAL continuously
archive_mode=on
archive_command='pgbackrest --stanza=main archive-push %p'
```

Mature tools such as **pgBackRest** and **Barman** manage full, differential and incremental backups, parallel transfer, compression, encryption, retention and verification, so you do not write that logic yourself. Monitor the age of the last successful backup and the lag of WAL archiving, because a quiet failure in `archive_command` makes recovery points disappear.

Recover to a point in time

```
$ pgbackrest --stanza=main --type=time "--target=2026-03-04 14:29:00+00" --target-action=promote r \
    estore
$ pg_ctl -D /var/lib/postgresql/data start
```

The restore fetches the base backup, then replays WAL up to the target time and stops. Choose the target just before the mistake, restore **to a new server first**, check the data, and then either switch to it or copy the missing rows back. Do not restore over the damaged primary until you know what you need. Note that recovery returns the database to an earlier state, so changes made after the target are lost unless you extract them.

MySQL and others

MySQL uses a full backup (for example with Percona XtraBackup) plus binary logs for point-in-time recovery. Cloud-managed databases offer automated backups and point-in-time restore with a retention window; know that window, test a restore and keep your own export for long retention and for protection against account problems.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: database backups and point-in-time... fits together

Part	What it does	What goes wrong
Logical or physical	A logical backup (pg_dump) exports schema and data as SQL or an archive.	Copying live data files and calling it a backup.
Archive the WAL	Every change goes to the WAL first. If you archive each WAL segment to safe storage, you can replay changes on top of a base backup up to any moment.	No alert when archiving fails.
Recover to a point in time	The restore fetches the base backup, then replays WAL up to the target time and stops.	Restoring over the damaged primary at once.
MySQL and others	MySQL uses a full backup (for example with Percona XtraBackup) plus binary logs for point-in-time recovery.	Never testing a point-in-time restore.

Flow: Logical or physical to Archive the WAL / Archive the WAL to Recover to a point in time / Recover to a point in time to MySQL and others

Incident: The archive that stopped in January

Incident: *During a recovery drill the team found that the last archived WAL segment was nine weeks old.*

```
$ pgbackrest --stanza=main info
wal archive min/max: 000000010000000A000000F2 / 000000010000000B0000003C
$ psql -c "select last_archived_time, last_failed_time from pg_stat_archiver"
 2026-01-09 03:11:42 | 2026-01-09 03:12:01
```

A credential for the archive bucket had been rotated, and `archive_command` had failed ever since. The database kept running and WAL files piled up until the disk nearly filled. Nobody had alerts on archive failures. The team fixed the credential, took a new full backup, added alerts on `pg_stat_archiver.last_failed_time` and the age of the newest backup, and made a quarterly restore drill part of the calendar.

Common mistakes

- Copying live data files and calling it a backup
- No alert when archiving fails
- Restoring over the damaged primary at once
- Never testing a point-in-time restore

Check yourself

What makes point-in-time recovery possible?

Answer: A base backup plus continuously archived write-ahead log

Where should you first restore after an accidental delete?

Answer: To a separate server, to inspect before switching

Which signal shows archiving is broken?

Answer: pg_stat_archiver shows recent failures or an old last_archived_time

When is pg_dump a good fit?

Answer: Portable backups of a single database or modest data size

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Backup and DR: Kubernetes, etcd and volume snapshots

12 minutes. Protect a Kubernetes cluster by backing up etcd, declarative manifests and persistent volumes, and know what each backup can and cannot restore.

A cluster is made of several things that fail differently: the control plane state in etcd, the manifests that describe your applications, the data in persistent volumes and the cloud resources around it. A good plan covers each of these and makes rebuilding the cluster from code a normal operation.

What you will learn

- What to protect in a cluster and in which layer
- Backing up and restoring etcd
- Volume snapshots with the CSI snapshot API
- Application-level backups and cluster rebuild strategy

Layers of a cluster

1. **Declarative config**: manifests, Helm values and Terraform in version control. This is the main source for rebuilding workloads, and it is why you do not rely on etcd for application recovery in most cases.
2. **Cluster state in etcd**: needed for self-managed control planes. Managed services (EKS, AKS, GKE) run it for you, so check what the provider backs up and what you must keep yourself.
3. **Persistent data**: volumes and external databases. This is the part that cannot be recreated from code.
4. **Secrets and keys**: encryption keys, certificates and external secret stores. Back these up in a safe way that works if the cluster is gone.

etcd snapshots

```
$ ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-2026-03-04.db --endpoints=https://127.0.0.1:237 \
    9 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/e \
    tc/kubernetes/pki/etcd/server.key
$ ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-2026-03-04.db --write-out=table
```

Take snapshots on a schedule (for example every 30 minutes), copy them off the node, encrypt them (they contain Secrets), and test restoring on a throwaway cluster. Restoring etcd on a multi-node control plane needs a documented procedure: stop the API servers, restore the same snapshot on every member with a new data directory and cluster settings, then restart. Practice it before you need it.

Volume snapshots

The CSI snapshot API takes storage-level snapshots of PersistentVolumeClaims, if the driver supports it.

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: orders-db-2026-03-04
  namespace: shop
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: orders-db-data
```

To restore, create a new PVC with `dataSource` pointing at the snapshot. A storage snapshot is **crash-consistent**: it looks like the machine lost power, which databases can usually recover from but may take time. For **application-consistent** backups, flush or quiesce first, or use the database's own backup tool and store the result off the cluster. Snapshots usually live in the same storage system, so copy important ones to another region or account.

MARGA by EknathaLabs marga.eknathalabs.com

Tools and rebuild strategy

Tools such as Velero back up namespaced resources and volumes to object storage and can restore to another cluster, which also helps migrations. A mature approach is: rebuild the cluster from infrastructure code, let your normal delivery pipeline reapply manifests from version control, then restore data volumes and databases from backups. Rehearse this end to end and measure the time.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: Kubernetes, etcd and volume... fits together

Part	What it does	What goes wrong
Layers of a cluster	Declarative config: manifests, Helm values and Terraform in version control.	Backing up etcd but never testing a restore.
etcd snapshots	Take snapshots on a schedule (for example every 30 minutes), copy them off the node, encrypt them (they contain Secrets), and test restoring on a throwaway cluster.	Storing etcd snapshots unencrypted on the node.
Volume snapshots	The CSI snapshot API takes storage-level snapshots of PersistentVolumeClaims, if the driver supports it.	Assuming a volume snapshot is application-consistent.
Tools and rebuild strategy	Tools such as Velero back up namespaced resources and volumes to object storage and can restore to another cluster, which also helps migrations.	Keeping snapshots only in the same storage system.

Flow: Layers of a cluster to etcd snapshots / etcd snapshots to Volume snapshots / Volume snapshots to Tools and rebuild strategy

Incident: The snapshot that restored a broken database

> **Incident:** *A team restored a volume snapshot of their database after a failed upgrade, and the database then needed hours of crash recovery and lost recent transactions.*

```
$ kubectl get volumesnapshot -n shop
NAME                    READYTOUSE  SOURCEPVC       CREATIONTIME
orders-db-2026-03-04    true        orders-db-data  3h12m
$ kubectl logs orders-db-0 | tail -3
database system was interrupted; last known up at 2026-03-04 09:47:12
redo starts at 0/5A0D2F8
```

The snapshot was taken while the database was writing, so it was crash-consistent and recovery had to replay a long log. The team had never practiced it and had no logical or WAL-based backup as a second path. Afterward they used the database's native backups with log archiving for data, kept snapshots as a fast first option, and added a pre-snapshot hook and a restore drill each quarter.

Common mistakes

- Backing up etcd but never testing a restore
- Storing etcd snapshots unencrypted on the node
- Assuming a volume snapshot is application-consistent
- Keeping snapshots only in the same storage system

Check yourself

What is the main source to rebuild stateless workloads?

Answer: Declarative manifests in version control

Why encrypt etcd backups?

Answer: They contain Secrets and cluster state

What does crash-consistent mean?

Answer: The volume looks like the machine lost power at that moment

How do you restore from a CSI VolumeSnapshot?

Answer: Create a new PVC whose dataSource points to the snapshot

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Backup and DR: testing restores and verifying backups

11 minutes. Prove that backups work with automated restore tests, integrity checks and timed drills, and track recovery time and success rate over time.

Nobody has a backup problem until they need to restore. A backup that has never been restored is a hope. The remedy is routine: restore on a schedule, check the result with real queries, record how long it took and alert when anything fails.

What you will learn

- Three levels of verification: job success, integrity and a real restore
- Building an automated restore test
- Measuring recovery time against your RTO
- Handling backup failures like production incidents

Three levels of proof

1. **The job finished**: necessary but weak. Jobs can succeed and write empty or damaged files.
2. **Integrity checks**: checksums, `restic check`, `pgbackrest verify`, comparing sizes and counts with expectations.
3. **Real restore and use**: restore into a clean environment, start the system, and run application-level checks, such as row counts, the latest timestamps and a known test query.

Only the third level proves you can recover. Run it regularly, and more often for critical systems.

```
$ restic -r s3:s3.amazonaws.com/example-backups check --read-data-subset=10%
$ pgbackrest --stanza=main verify
$ sha256sum -c /backup/MANIFEST.sha256
```

An automated restore test

```
#!/usr/bin/env bash
set -euo pipefail
start=$(date +%s)
workdir=$(mktemp -d)
trap 'rm -rf "$workdir"' EXIT

restic -r "$REPO" restore latest --target "$workdir" --tag nightly
count=$(find "$workdir" -type f | wc -l)
newest=$(find "$workdir" -type f -printf '%T@\n' | sort -n | tail -1 | cut -d. -f1)
age=$(( $(date +%s) - newest ))

[ "$count" -gt 1000 ] || { echo "too few files: $count"; exit 1; }
[ "$age" -lt 172800 ] || { echo "newest file is older than 48h"; exit 1; }
echo "restore ok: $count files, $(( $(date +%s) - start ))s"
```

Run it from a scheduler, push the duration and result to your metrics system, and alert on failure and on a restore time that grows beyond a fraction of the RTO (for example 80 percent). For databases, restore into a scratch instance and run a query set that compares counts and latest dates with production.

Timed drills

Beyond automation, run a drill where a person follows the written runbook to restore a service in a clean environment, with a clock. This tests the human parts: access, credentials, documentation and decisions. Record steps that were unclear or missing, fix them, and repeat. Include a scenario where the person who usually does this is unavailable.

MARGA by EknathaLabs marga.eknathalabs.com

When a backup fails

Treat a failed or missing backup as an incident of its own priority: alert, investigate, fix and take a new backup soon, instead of waiting for the next window. Track success rate, backup age, restore time and the number of systems with a tested restore in the last quarter, and report them to the owners.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: testing restores and verifying... fits together

Part	What it does	What goes wrong
Three levels of proof	The job finished: necessary but weak. Jobs can succeed and write empty or damaged files.	Testing restores with no application-level checks.
An automated restore test	Run it from a scheduler, push the duration and result to your metrics system, and alert on failure and on a restore time that grows beyond a fraction of the RTO (for...	Never timing a restore against the RTO.
Timed drills	Beyond automation, run a drill where a person follows the written runbook to restore a service in a clean environment, with a clock.	Check the Common mistakes list below this diagram for what usually goes wrong here.
When a backup fails	Treat a failed or missing backup as an incident of its own priority: alert, investigate, fix and take a new backup soon, instead of waiting for the next window.	Restoring only after a disaster.

Flow: Three levels of proof to An automated restore test / An automated restore test to Timed drills / Timed drills to When a backup fails

Incident: Nine months of empty backups

> **Incident:** *When a file server failed, the team discovered that the nightly backups had been empty since a path change nine months earlier.*

```
$ ls -la /backup/fileserver/2026-02-28/
total 8
drwxr-xr-x 2 backup backup 4096 Feb 28 02:10 .
drwxr-xr-x 9 backup backup 4096 Feb 28 02:10 ..
$ grep -c "completed successfully" /var/log/backup.log
271
```

The job pointed at a directory that no longer existed, and the tool treated an empty source as success. Each night the log said "completed successfully". The business recovered part of the data from user laptops. The team added a check on backup size against a baseline, a weekly automated restore test with file counts and a quarterly drill, and a dashboard showing the last tested restore for every system.

Common mistakes

- Treating "job succeeded" as proof
- Restoring only after a disaster
- Testing restores with no application-level checks
- Never timing a restore against the RTO

Check yourself

Which test proves a backup is recoverable?

Answer: Restoring it to a clean environment and using the data

Why track restore duration?

Answer: To compare it with the RTO and see growth as data grows

MARGA by EknathaLabs marga.eknathalabs.com

What should a restore test check beyond the file count?

Answer: Application-level facts such as row counts and latest timestamps

How should you react to a failed backup?

Answer: Treat it as an incident and take a new backup soon

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Backup and DR: disaster recovery patterns and failover

12 minutes. Compare backup and restore, pilot light, warm standby and active-active designs, and plan data replication, DNS failover and failback for each recovery objective.

Disaster recovery (DR) is the ability to run your service somewhere else after losing a site, a region or an account. The right design depends on how much downtime and data loss the business accepts and how much it will pay. This lesson compares the standard patterns and the details that decide whether they work.

What you will learn

- The four common DR patterns and their trade-offs
- Replicating data, and why order and consistency matter
- Traffic failover with DNS and load balancers
- Failback and keeping the DR site honest

Four patterns

1. **Backup and restore**: copies of data and the code to rebuild live elsewhere. Cheapest, with an RTO of hours to days.
2. **Pilot light**: the core data layer runs and replicates continuously in the DR site, while application servers are off or minimal and are started on demand. RTO is tens of minutes to hours.
3. **Warm standby**: a smaller but complete copy runs all the time and scales up on failover. RTO is minutes.
4. **Active-active** (multi-site): several sites serve live traffic. RTO near zero, with the highest cost and the hardest data consistency problems.

Pick per service. A payment system may justify warm standby or active-active. An internal wiki can use backup and restore.

Data is the hard part

Stateless servers are easy to rebuild. Data needs replication: asynchronous replication keeps latency low but can lose the last seconds on failover (non-zero RPO), while synchronous replication across distance adds latency to every write and can reduce availability. Decide RPO per data store, and ensure related data fails over together (a database and its object storage, a queue and its consumers), or you will see orders without their files. Think about secrets, certificates, container images and configuration, which must exist in the DR site before the disaster, not be fetched from the lost site.

Moving traffic

```
$ dig +short app.example.com
$ aws route53 list-health-checks --max-items 3
$ curl -s -o /dev/null -w '%{http_code}\n' https://dr.example.com/health
```

Common methods: DNS failover with health checks and a short TTL (some clients cache longer than the TTL), global load balancers or anycast addresses (faster, provider specific), and manual switching steps in a runbook. Decide who is allowed to trigger failover and with which evidence, since a false failover causes its own outage and a split-brain risk if both sites accept writes.

MARGA by EknathaLabs marga.eknathalabs.com

Failback and keeping DR honest

After the primary returns, plan the way back: re-sync data in the reverse direction, switch traffic in a quiet window and then re-establish the original protection. Keep the DR site tested and in step with production: run infrastructure from the same code, deploy to both, use game days to fail over real traffic for non-critical services, and compare capacity so the DR site can carry the load. A DR site that is a year behind production will fail when you need it.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: disaster recovery patterns and... fits together

Part	What it does	What goes wrong
Four patterns	Backup and restore: copies of data and the code to rebuild live elsewhere.	Replicating data but not secrets, images and configuration.
Data is the hard part	Stateless servers are easy to rebuild. Data needs replication: asynchronous replication keeps latency low but can lose the last seconds on failover (non-zero RPO), while...	A DR site that drifts from production or lacks capacity and quota.
Moving traffic	Common methods: DNS failover with health checks and a short TTL (some clients cache longer than the TTL), global load balancers or anycast addresses (faster, provider...	No decision rule for who triggers failover.
Failback and keeping DR honest	After the primary returns, plan the way back: re-sync data in the reverse direction, switch traffic in a quiet window and then re-establish the original protection.	A DR site that is a year behind production will fail when you need it.

Flow: Four patterns to Data is the hard part / Data is the hard part to Moving traffic / Moving traffic to Failback and keeping DR honest

Incident: The standby that could not take the load

> **Incident:** *When a region failed, the standby in the second region started but ran out of capacity within minutes and returned errors.*

```
$ kubectl get nodes --context dr-cluster | wc -l
4
$ kubectl get nodes --context prod-cluster | wc -l
36
```

The DR cluster had been sized for "a small copy" and had a quota limit on instance launches that nobody had raised. Autoscaling hit the quota after three nodes. Failover succeeded technically but the service was degraded for 90 minutes. The team raised quotas, pre-provisioned capacity for the critical tier, ran a load test on the DR site each quarter and added the DR capacity to the architecture review.

Common mistakes

- Choosing a pattern without agreed RTO and RPO
- Replicating data but not secrets, images and configuration
- No decision rule for who triggers failover
- A DR site that drifts from production or lacks capacity and quota

Check yourself

Which pattern has a continuously replicating data layer but minimal application servers?

Answer: Pilot light

What is the RPO impact of asynchronous replication?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Some recent writes may be lost during failover

Why decide in advance who triggers a failover?

Answer: A wrong or late decision can cause an outage or split-brain

What often limits a DR site at failover?

Answer: Capacity and quota that were never raised or tested

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Backup and DR: ransomware, immutable backups and recovery runbooks

12 minutes. Defend backups against deletion and encryption with immutability, separate accounts and access control, and write a runbook people can follow under pressure.

Attackers know that backups are the last line of defense, so they try to find and destroy them first. A modern backup design assumes that an administrator account may be compromised, and keeps at least one copy that nobody can change or delete during its retention period. Preparing the recovery runbook before the event matters just as much as the technology.

What you will learn

- How attacks target backups and what stops them
- Immutability, object lock and air gaps
- Separate accounts, least privilege and monitoring
- A recovery runbook and an order of restoration

What attackers do

A typical attack gains credentials, spreads, steals data, deletes or encrypts backups and then encrypts production. They look for backup consoles, backup servers joined to the same directory, storage buckets with write and delete rights, and snapshots in the same account. Defenses must therefore break the link between "can reach production" and "can destroy backups".

Immutable and isolated copies

- **Immutability / object lock / WORM**: the storage refuses deletes and overwrites until a retention date, even for administrators in the retention-lock mode. Cloud object stores, backup appliances and tape libraries support this.
- **Separate account or provider**: backups live in an account with different credentials and a separate identity, reached only by a narrow write-only role from production.
- **Air gap**: offline media or a logically isolated vault that is not reachable during normal operations.
- **Versioning plus MFA delete** and long enough retention that you can go back before the attacker arrived. Attackers often wait weeks.

```
$ aws s3api put-bucket-versioning --bucket dr-vault-backups --versioning-configuration Status=Enab \
    led
$ aws s3api put-object-lock-configuration --bucket dr-vault-backups --object-lock-configuration '{ \
   "ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":{"Mode":"COMPLIANCE","Days":35}}}'
$ aws s3api get-object-lock-configuration --bucket dr-vault-backups
```

Object lock is normally enabled when the bucket is created, and compliance mode cannot be shortened by anyone, so choose retention carefully. Alert on any delete, policy change or failed backup, and keep the backup system's own admin accounts under separate, strongly authenticated control.

The recovery runbook

Write it before you need it, store it where it is reachable when systems are down (printed copy, offline copy, separate system), and include:

1. **Who decides and who does**: incident lead, communications lead and technical leads with phone numbers that do not depend on company systems.
2. **Contain first**: isolate affected systems and accounts, rotate credentials, and preserve evidence for investigation. Restoring into an environment that is still compromised wastes the restore.
3. **Order of restoration**: identity and DNS, networking, secrets and certificates, data stores, then applications, by dependency and business priority.

MARGA by EknathaLabs marga.eknathalabs.com

4 **Clean restore points**: how to choose a backup from before the compromise, and how to scan restored data.

5 **Verification and sign-off**: checks that each service is correct before it takes traffic.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Backup and DR: ransomware, immutable backups and... fits together

Part	What it does	What goes wrong
What attackers do	A typical attack gains credentials, spreads, steals data, deletes or encrypts backups and then encrypts production.	Backups reachable with the same credentials as production.
Immutable and isolated copies	Immutability / object lock / WORM: the storage refuses deletes and overwrites until a retention date, even for administrators in the retention-lock mode.	Retention shorter than the time attackers stay hidden.
The recovery runbook	Write it before you need it, store it where it is reachable when systems are down (printed copy, offline copy, separate system), and include.	A runbook that exists only on systems that are down.

Flow: What attackers do to Immutable and isolated copies / Immutable and isolated copies to The recovery runbook

Incident: Restoring into a compromised network

Incident: *After a ransomware attack, a company restored its servers from backup within a day, and they were encrypted again two hours later.*

```
$ last -n 3
admin pts/0 203.0.113.55 Mar 4 01:12 still logged in
$ crontab -l -u svc-backup
*/5 * * * * /tmp/.cache/agent
```

The attacker's persistence (a stolen service account and a scheduled task) was still in place. Backups were good, but the environment was not clean. The company rebuilt identity and rotated all credentials first, restored into a new network segment from a point before the first compromise, scanned the data and brought services back in dependency order. Their new runbook starts with containment and has checklists for each step.

Common mistakes

- Backups reachable with the same credentials as production
- Retention shorter than the time attackers stay hidden
- Restoring before containing the intrusion
- A runbook that exists only on systems that are down

Check yourself

What does object lock in compliance mode provide?

Answer: Objects cannot be deleted or overwritten until retention ends, even by admins

Why use a separate account for backups?

Answer: Production credentials then cannot delete or alter them

What should come first in a ransomware recovery?

Answer: Contain the intrusion and secure identity before restoring

Where should the recovery runbook be stored?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Somewhere reachable when company systems are down, including offline

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Storage benchmarking with fio

12 minutes. Measure disks and volumes with fio using realistic block sizes, queue depths and read-write mixes, and read latency percentiles correctly.

Disks are described by throughput numbers that rarely match application behavior. A database does small random reads and writes, while backups stream large files. fio lets you reproduce both and see the difference.

What you will learn

- What IOPS, bandwidth and latency mean
- Writing fio jobs for common workloads
- Reading percentile latency
- Safe testing practices

The basics

IOPS counts operations per second, **bandwidth** is bytes per second and **latency** is the time per operation. They depend on block size and queue depth. Small random operations stress IOPS and latency, while large sequential operations stress bandwidth.

Jobs for real workloads

```
; db-like.fio  - random 8k, 70% reads, queue depth 32
[db-like]
filename=/mnt/scratch/fio.dat
size=4G
direct=1
ioengine=libaio
rw=randrw
rwmixread=70
bs=8k
iodepth=32
numjobs=4
time_based=1
runtime=60
group_reporting=1

fio db-like.fio
fio --name=seqread --filename=/mnt/scratch/fio.dat --size=4G --direct=1 \
    --rw=read --bs=1M --iodepth=8 --runtime=30 --time_based
```

`direct=1` bypasses the page cache, otherwise you measure memory. Match `bs`, `iodepth` and the read-write mix to your application. Run longer than the time a burst allowance lasts on cloud volumes.

Reading the result

```
read: IOPS=11.2k, BW=87.5MiB/s
  clat percentiles (usec):
   | 50.00th=[  620], 95.00th=[ 1860], 99.00th=[ 7300], 99.90th=[21100]
```

Look at the **percentiles**, especially the 99th and 99.9th. Averages hide the slow tail that users feel. Compare results with the limits of the volume type, and watch `iostat -x` during the test.

Safety

Never run write tests on a device holding data. Use a scratch file or a new volume, and delete it afterwards. Test during maintenance windows on shared storage, since heavy tests affect neighbours.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Storage benchmarking with fio fits together

Part	What it does	What goes wrong
The basics	IOPS counts operations per second, bandwidth is bytes per second and latency is the time per operation.	Only measuring sequential throughput.
Jobs for real workloads	direct=1 bypasses the page cache, otherwise you measure memory.	Testing without direct I/O.
Reading the result	Look at the percentiles, especially the 99th and 99.9th.	Reading averages and ignoring the tail.
Safety	Never run write tests on a device holding data.	Writing to a device that holds data.

Flow: The basics to Jobs for real workloads / Jobs for real workloads to Reading the result / Reading the result to Safety

Incident: Fast in the test, slow in production

Incident: *A sequential test hid poor random latency.*

```
seqread: BW=1.2GiB/s
randread 4k, iodepth=1: p99 latency 14ms
database commit latency: 19ms
```

The team had chosen a throughput-optimized volume after a large-file test. The database needed low-latency random I/O. They switched to a volume type designed for IOPS and confirmed it with a random test that matched the database settings.

Common mistakes

- Testing without direct I/O
- Only measuring sequential throughput
- Reading averages and ignoring the tail
- Writing to a device that holds data

Check yourself

What does direct=1 do?

Answer: Bypasses the page cache

Which metric shows the slow tail?

Answer: 99th percentile latency

What stresses IOPS?

Answer: Small random operations

Where run write tests?

Answer: On a scratch volume

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Storage and filesystems

Disks, filesystems, LVM, RAID, network storage and the commands that explain a full, slow or missing volume.

See what is there

```
lsblk -f
blkid
df -hT
df -i
findmnt
mount | column -t
du -xh --max-depth=1 /var | sort -h | tail
```

Warning: `df` *and* `du` *can disagree. A deleted file that a process still holds open uses space until it closes.*

```
lsof +L1
```

Filesystems

```
mkfs.ext4 /dev/sdb1
mkfs.xfs /dev/sdb1
mount /dev/sdb1 /data
umount /data
tune2fs -l /dev/sdb1 | head
xfs_info /data
fsck -n /dev/sdb1     # check, no changes
```

fstab line:

```
UUID=7c1d...  /data  xfs  defaults,nofail  0  0
```

Test with `mount -a` before you reboot.

LVM

```
pvs; vgs; lvs
pvcreate /dev/sdc
vgextend vg0 /dev/sdc
lvextend -r -L +20G /dev/vg0/data   # -r grows the filesystem too
resize2fs /dev/vg0/data             # ext4
xfs_growfs /data                    # XFS, takes the mount point
```

XFS can grow but not shrink.

RAID

```
cat /proc/mdstat
mdadm --detail /dev/md0
mdadm /dev/md0 --add /dev/sdd1
```

Performance

```
iostat -x 1
iotop -o
vmstat 1
```

Watch `%util`, `await` and queue size, not only throughput.

Network storage

```
showmount -e server
mount -t nfs4 server:/export /mnt
nfsstat -m
iscsiadm -m session
ceph status
```

MARGA by EknathaLabs
marga.eknathalabs.com

Quick diagnosis

Symptom	First look
Full but nothing to delete	`df -i`, `lsof +L1`
Read-only filesystem	`dmesg`, `mount`, disk errors
Slow	`iostat -x`, queue and latency
Boot stops after mount change	`journalctl -xb`, fstab UUID, `nofail`

Cautions

Danger: `mkfs` *and* `wipefs` *destroy the data on a device. Check the name with* `lsblk` *and* `blkid` *every time.*

Warning: *Shrinking a filesystem or logical volume in the wrong order loses data. XFS cannot shrink at all.*

Warning: *A wrong line in /etc/fstab can stop boot. Use UUIDs and* `nofail`*, and run* `mount -a` *before rebooting.*

Caution: *RAID is not a backup. A deleted file is deleted on every disk.*

MARGA by EknathaLabs
marga.eknathalabs.com